Programming Fundamentals LAB – F25

Lab 10 – 18-11-2025

In drive D: or some other suitable drive create a folder pflab07 and change folder to that so that the prompt should be like **D:\rollno\pflab09>.**

Make sure that command g++ is recognized at the computer you have, also check the Notepad++.

To create or edit abc.cpp, use command: **D:\rollno\pflab09>**`start notepad++ abc.cpp`
To compile abc.cpp, use command: **D:\rollno\pflab09>**`g++ abc.cpp -o abc.exe`
To run binary of abc.cpp, the abc.exe, use command: **D:\rollno\pflab09>**`abc.exe`

This is a take home LAB due to convocation. The following are not Lab tasks for this Lab.

You just need to completely understand the code attached for a related viva in coming Lab. You may practice similar problems by your own

Stay Blessed.

Thank you for your **patience**

```
#include <iostream>
#include <string>
#include <cmath> // Required for the sqrt and pow functions

using namespace std;

// 1. Define the Point structure
struct Point
{
    double x; // x-coordinate
    double y; // y-coordinate
};

// 2. Function to calculate the distance between two Point structures
/**
 * Calculates the Euclidean distance between two points (p1 and p2).
 * The formula used is the distance formula:
 * distance = sqrt((x2 - x1)^2 + (y2 - y1)^2)
 */
double calculateDistance(Point p1, Point p2)
{
```

```
    // Calculate the difference in x-coordinates (dx)
    double dx = p2.x - p1.x;

    // Calculate the difference in y-coordinates (dy)
    double dy = p2.y - p1.y;

    // Calculate (dx)^2 + (dy)^2
    double distanceSquared = pow(dx, 2) + pow(dy, 2);

    // Take the square root of the sum
    return sqrt(distanceSquared);
}

// 3. Main function
int main()
{
    // Create two Point objects
    Point point1;
    Point point2;

    // --- Input for Point 1 ---
    cout << "Enter the coordinates for Point 1 (x y): ";
    // Use cin to read the x and y values directly into the struct members
    cin >> point1.x >> point1.y;

    // --- Input for Point 2 ---
    cout << "Enter the coordinates for Point 2 (x y): ";
    cin >> point2.x >> point2.y;

    // 4. Calculate the distance using the function
    double distance = calculateDistance(point1, point2);

    // 5. Output the result
    cout << "\n--- Results ---" << endl;
    cout << "Point 1: (" << point1.x << ", " << point1.y << ")" << endl;
    cout << "Point 2: (" << point2.x << ", " << point2.y << ")" << endl;
    cout << "The distance between the two points is: " << distance << endl;

    return 0;
}
```

```
#include <iostream>
#include <string>
#include <cmath>
#include <array>

using namespace std;

// 1. Define the Point structure
struct Point
{
    double x;
    double y;
};

// --- Type Definitions for Clarity ---
enum TriangleType
{
    EQUILATERAL,     // three sides with same lengths
```

```
    ISOSCELES,       // two sides with same lengths
    SCALENE,         // all sides with different lengths
    RIGHT_ISOSCELES, // New type: Right Angle AND two equal sides
    RIGHT_SCALENE,   // New type: Right Angle AND three different sides
    NOT_A_TRIANGLE
};

// A small constant for robust floating-point comparison
const double EPSILON = 0.00000001;

// 2. Function to calculate the squared distance (useful for Pythagorean theorem check)
double calculateDistanceSquared(const Point& p1, const Point& p2)
{
    double dx = p2.x - p1.x;
    double dy = p2.y - p1.y;
    // We return the squared distance: (dx)^2 + (dy)^2
    return pow(dx, 2) + pow(dy, 2);
}

// 3. Helper function for floating-point comparison
bool areEqual(double a, double b)
{
    return abs(a - b) < EPSILON;
}

// 4. Function to determine the type of triangle
TriangleType getTriangleType(array<Point, 3> points)
{
    // a. Calculate the SQUARED length of the three sides
    // This avoids sqrt() calls until needed and is necessary for Pythagoras check.
    double sideASq = calculateDistanceSquared(points[0], points[1]);
    double sideBSq = calculateDistanceSquared(points[1], points[2]);
    double sideCSq = calculateDistanceSquared(points[2], points[0]);

    // Calculate the actual side lengths (distance) for the Inequality Theorem
    double sideA = sqrt(sideASq);
    double sideB = sqrt(sideBSq);
    double sideC = sqrt(sideCSq);

    // b. Collinear Check (Not a Triangle) using the Triangle Inequality Theorem
    if (!((sideA + sideB > sideC + EPSILON) &&
          (sideA + sideC > sideB + EPSILON) &&
          (sideB + sideC > sideA + EPSILON)))
    {
        return NOT_A_TRIANGLE;
    }

    // c. Determine equality of sides
    bool isAEqualB = areEqual(sideA, sideB);
    bool isBEqualC = areEqual(sideB, sideC);
    bool isAEqualC = areEqual(sideA, sideC);

    int equalSidesCount = 0;
    if (isAEqualB)
    {
        equalSidesCount++;
    }
    if (isBEqualC)
    {
        equalSidesCount++;
    }
    if (isAEqualC)
```

```
    {
        equalSidesCount++;
    }

    bool isEquilateral = (equalSidesCount == 3);
    bool isIsosceles = (equalSidesCount >= 1); // Equilateral is also Isosceles by
definition
    bool isScalene = (equalSidesCount == 0);

    // d. Check for Right Angle using Pythagorean Theorem (a^2 + b^2 = c^2)
    // Use the squared side lengths for this check.
    bool isRightAngle = areEqual(sideASq + sideBSq, sideCSq) ||
                        areEqual(sideBSq + sideCSq, sideASq) ||
                        areEqual(sideASq + sideCSq, sideBSq);

    // e. Final Classification

    // Case 1: Right Triangle check
    if (isRightAngle)
    {
        if (isAEqualB || isBEqualC || isAEqualC)
        {
            // Two sides are equal AND it has a right angle
            return RIGHT_ISOSCELES;
        }
        else
        {
            // Three sides are different AND it has a right angle
            return RIGHT_SCALENE;
        }
    }

    // Case 2: Non-Right Triangle (Acute or Obtuse)
    if (isEquilateral)
    {
        return EQUILATERAL;
    }

    if (isIsosceles)
    {
        return ISOSCELES;
    }

    // If none of the above, it must be Scalene (and not Right)
    return SCALENE;
}

// 5. Main function
int main()
{
    array<Point, 3> trianglePoints;

    cout << "--- Advanced Triangle Type Checker ---" << endl;

    // Input the three points from the user
    for (int i = 0; i < 3; i=i+1)
    {
        cout << "Enter the coordinates for Point " << i + 1 << " (x y): ";
        cin >> trianglePoints[i].x >> trianglePoints[i].y;
    }

    // 6. Determine the type of triangle
```

```
    TriangleType type = getTriangleType(trianglePoints);

    // 7. Output the result
    cout << "\n--- Result ---" << endl;
    cout << "The points form a ";

    if (type == EQUILATERAL) {
        cout << "**Equilateral** triangle (3 equal sides, 60 degree angles)." << endl;
    } else if (type == ISOSCELES) {
        cout << "**Isosceles** triangle (2 equal sides, no right angle)." << endl;
    } else if (type == SCALENE) {
        cout << "**Scalene** triangle (3 different sides, no right angle)." << endl;
    } else if (type == RIGHT_ISOSCELES) {
        cout << "**Right Isosceles** triangle (Right angle AND 2 equal sides, 45, 45, and
90 degrees)." << endl;
    } else if (type == RIGHT_SCALENE) {
        cout << "**Right Scalene** triangle (Right angle AND 3 different sides)." << endl;
    } else if (type == NOT_A_TRIANGLE) {
        cout << "**NOT A TRIANGLE** (Points are collinear or identical)." << endl;
    }

    return 0;
}
```